



MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul

Otomatisasi Prediktif: Supervised

Abstrak

Pertemuan ini memperkenalkan machine learning sebagai pelengkap monitoring rule-based (Pertemuan 12-13) untuk

Sub - CPMK

Sub-CPMK 5.1 – Mampu melakukan pengukuran kinerja sistem dan menganalisis hasilnya

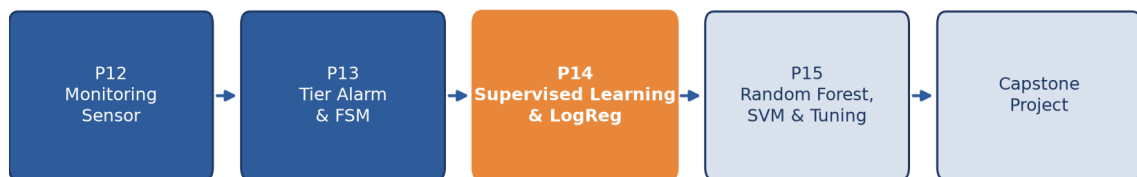
Disusun Oleh :
Dedik Romahadi, ST., M.Sc

 **Modul Interaktif:** <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-13.html>. Pada layar masuk, pilih tombol “ Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Rule-based monitoring (Pertemuan 12-13) sangat transparan dan reliable, tetapi memiliki tiga keterbatasan: hanya bekerja baik untuk batas keputusan linier (single/few threshold), threshold tetap 0,5 tidak selalu optimal, dan satu kali split data rawan bias. Pertemuan keempat belas ini memperkenalkan machine learning sebagai pelengkap: model belajar pola dari data historis berlabel, dan mampu menangani kombinasi banyak fitur sekaligus yang sulit ditangani rule tunggal.

Posisi Pertemuan 14 dalam Alur Mata Kuliah Optimalisasi & Otomasi



Gambar 1. Posisi Pertemuan 14 (Supervised Learning & Logistic Regression) dalam alur mata kuliah, mengawali blok materi machine learning sebelum Pertemuan 15.

Gambar 1 menunjukkan posisi Pertemuan 14 dalam alur mata kuliah.

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 5.1:** Mampu melakukan pengukuran kinerja sistem dan menganalisis hasilnya, yaitu mengukur kinerja model klasifikasi (accuracy, precision, recall, F_1) dan menganalisis hasilnya untuk menentukan kelayakan model pada sistem prediktif (CPMK 5).
- Setelah pertemuan ini mahasiswa dapat: membangun dataset X/y dan melakukan train/test split stratified; menjelaskan cara kerja Logistic Regression (sigmoid, decision boundary); menginterpretasikan confusion matrix; serta menghitung precision, recall, dan F_1 -score.

SUPERVISED LEARNING FOUNDATIONS: DATASET, FEATURES, LABELS & TRAINING WORKFLOW

$X[n,d], y[n]$ raw signal $\rightarrow$ 20+ fitur `train_test_split(X,y,0.2)`
`model.fit(X,y).predict(X_baru)`

Tabel 1. Tujuh tahap workflow supervised learning, dari load data hingga deploy ke production.

Tahap Workflow	Aksi	scikit-learn API	Output
1. Load data	Read CSV/Parquet $\rightarrow$ DataFrame	<code>pd.read_csv()</code>	X, y arrays
2. Preprocess	Standardize, encode, impute	<code>StandardScaler</code> , <code>OneHotEncoder</code>	X_{scaled}
3. Split	Train/test 80/20 stratified	<code>train_test_split(stratify=y)</code>	$X_{\text{train}}, X_{\text{test}}, y_{\text{train}}, y_{\text{test}}$
4. Train	Fit model di train	<code>model.fit(X_train, y_train)</code>	Trained model object
5. Evaluate	Predict + metrics di test	<code>classification_report</code>	Accuracy, F_1 , confusion matrix
6. Tune	Grid search hyperparams	<code>GridSearchCV(cv=5)</code>	Best model + params
7. Deploy	Save model, integrasi API	<code>joblib.dump(model)</code>	File .pkl untuk production

Tabel 1 merangkum tujuh tahap workflow supervised learning standar menggunakan scikit-learn.

Stratified Train/Test Split: Proporsi Kelas Dipertahankan

Rasio kelas 90:10 dipertahankan di kedua split (`stratify=y`)



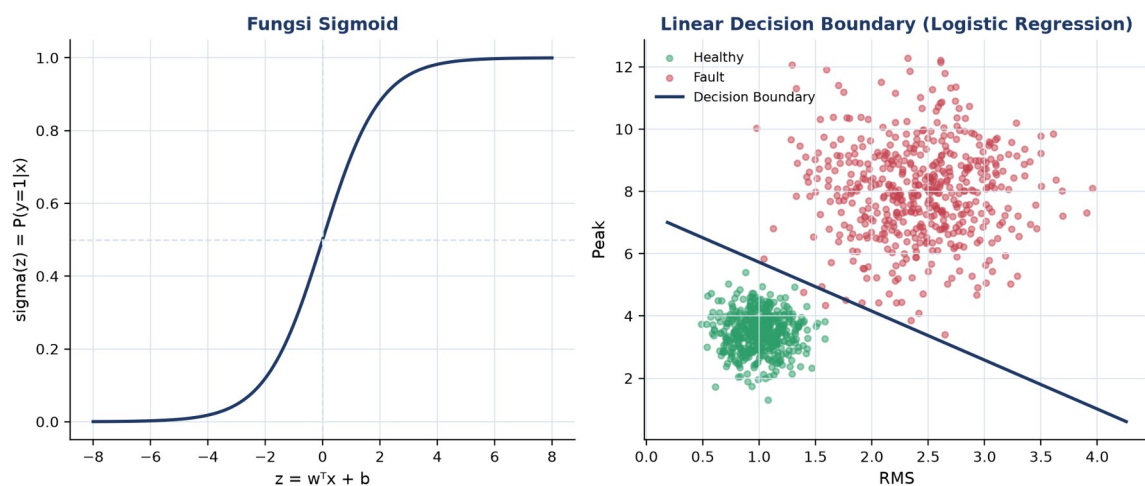
Gambar 2. Stratified train/test split: proporsi kelas 90:10 (healthy:fault) dipertahankan baik di train set (80%) maupun test set (20%).

Gambar 2 menunjukkan pentingnya parameter `'stratify=y'` pada `'train_test_split'`: tanpa stratify, test set berisiko kebetulan tidak memiliki sampel kelas minoritas sama sekali, terutama pada data yang sangat imbalanced.

Contoh Konkret: Bearing Fault Detection (CWRU Dataset). Benchmark CWRU (Case Western Reserve University) bearing dataset: 4 kondisi (healthy, inner race, outer race, ball fault), sekitar 10.000 sampel per kondisi, 21 fitur diekstrak dari gelombang mentah 50.000 titik. Random Forest 100 trees mencapai accuracy 98,5% dan $F_1=0,97$ (macro-average), jauh mengungguli pendekatan rule-based (sekitar 75% akurasi). Hubungan ini dirangkum dalam Persamaan (1).

LOGISTIC REGRESSION: LINEAR BASELINE CLASSIFIER UNTUK HEALTHY/FAULT DETECTION

$$P(y=1|x) = \sigma(w^T x + b) \quad \sigma(z) = 1/(1+e^{-z}) \quad \text{cross-entropy loss} \quad \text{koefisien } w = \log\text{-odds} \quad (1)$$



Gambar 3. Fungsi sigmoid mengubah kombinasi linier $z=w^T x+b$ menjadi probabilitas 0-1 (kiri); decision boundary linier Logistic Regression memisahkan healthy dan fault pada fitur RMS-Peak (kanan).

Gambar 3 menunjukkan bagaimana fungsi sigmoid memetakan kombinasi linier fitur menjadi probabilitas, dan bagaimana decision boundary yang dihasilkan berbentuk garis lurus (linier) pada ruang fitur; koefisien w dapat diinterpretasikan sebagai log-odds, memberi Logistic Regression keunggulan interpretability dibanding model kotak-hitam.

Tabel 2. Perbandingan empat algoritma klasifikasi: kecepatan training/prediksi, interpretability, dan kecocokan data. Random Forest dan SVM dibahas mendalam di Pertemuan 15.

Algoritma	Train Speed	Predict Speed	Interpretability	Cocok Data
Logistic Regression	Sangat cepat	Sangat cepat	Tinggi (koefisien w)	Linear separable, <100K
Random Forest	Sedang	Cepat	Sedang (feature_imp)	Mixed, <1M, baseline terbaik
SVM (RBF)	Lambat $O(n^2)$	Sedang	Rendah	Small-medium, high-dim
XGBoost / LightGBM	Sedang	Cepat	Sedang (SHAP)	Large tabular, Kaggle-grade

Tabel 2 membandingkan Logistic Regression dengan tiga algoritma lain yang akan dibahas lebih dalam di Pertemuan 15.

Contoh Konkret: Bearing Fault Classification (CWRU). Pada benchmark CWRU 4-kelas (21 fitur, 8000 train + 2000 test): Logistic Regression mencapai accuracy 86% dan $F_1=0,84$; Random Forest (default) accuracy 98,5% dan $F_1=0,97$; SVM RBF ($C=10$, $\gamma=scale$) accuracy 97,2% dan $F_1=0,96$; XGBoost (default) accuracy 98,8% dan $F_1=0,98$. Logistic Regression tertinggal dari model non-linear, tetapi tetap menjadi baseline wajib karena kesederhanaan dan interpretabilitasnya. Bentuk ringkasnya dituliskan pada Persamaan (2).

EVALUASI DASAR: CONFUSION MATRIX, ACCURACY, PRECISION, RECALL & F₁-SCORE

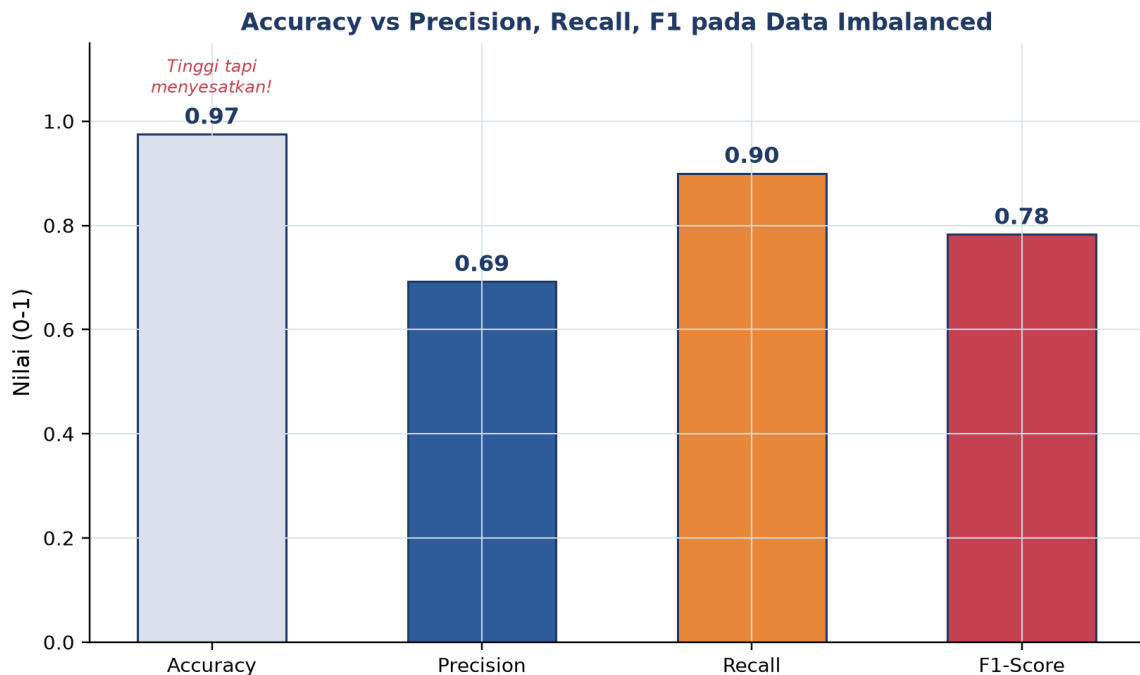
*Confusion matrix 2x2: $[[TN, FP], [FN, TP]]$ $Accuracy=(TP+TN)/N$
 $Precision=TP/(TP+FP)$ $Recall=TP/(TP+FN)$ $F_1=2PR/(P+R)$ (2)*

Confusion Matrix: Bearing Fault (Imbalanced, 5% Positif)

Actual	Healthy	TN 9300	FP 200
	Fault	FN 50	TP 450
		Predicted Healthy	Predicted Fault

Gambar 4. Confusion matrix studi kasus bearing fault imbalanced (5% positif): $TN=9300$, $FP=200$, $FN=50$, $TP=450$.

Gambar 4 menunjukkan confusion matrix untuk dataset dengan 9500 sampel healthy dan 500 sampel fault (rasio imbalance 19:1).



Gambar 5. Accuracy (0,975) tampak sangat tinggi tetapi menyesatkan pada data imbalanced; precision (0,69), recall (0,90), dan F1 (0,78) memberi gambaran lebih jujur tentang kinerja model.

Gambar 5 membuktikan mengapa accuracy saja tidak cukup untuk data imbalanced: naive predictor yang selalu memprediksi "healthy" sudah mencapai accuracy 95% padahal gagal total mendeteksi fault (recall=0%). Precision, recall, dan F₁-score memberi gambaran yang jauh lebih jujur tentang kinerja model pada kelas minoritas yang justru paling penting.

Peringatan, Trap Evaluasi yang Sering Diabaikan: Tujuh jebakan evaluasi yang sering diabaikan: memakai accuracy untuk data imbalanced; hanya mengandalkan satu kali split (variance tinggi); data leakage (informasi test bocor ke proses training); menggunakan ulang test set berkali-kali untuk tuning; distribusi data training tidak sama dengan kondisi deployment (concept drift); memakai threshold default 0,5 tanpa pertimbangan biaya kesalahan; serta melaporkan metrik yang salah untuk konteks masalah.

ANALOGI MACHINE LEARNING DI SEHARI-HARI

- **Email Spam Filter (Binary Classification):** Gmail mencapai akurasi lebih dari 99,9% dengan feature engineering dari kata kunci, link, dan reputasi pengirim yang diumpankan ke classifier biner.

- **Rekomendasi YouTube (Multi-Class + Regression):** YouTube menggabungkan collaborative filtering dan content-based filtering untuk memprediksi watch time dan merekomendasikan video.
- **Face Detection di Kamera (Computer Vision):** Viola-Jones (2001) memakai Haar cascade dan AdaBoost untuk deteksi wajah real-time; pendekatan modern beralih ke CNN yang dilatih pada jutaan foto.
- **Bank Credit Scoring (Logistic Regression Klasik):** skor kredit FICO memakai Logistic Regression yang diregulasi karena butuh interpretability tinggi (Fair Lending Act).
- **Diagnosis Medis ML (Decision Tree + Ensemble):** IBM Watson Health dan Google DeepMind Health memakai ensemble XGBoost dan deep learning untuk diagnosis medis, meski persetujuan FDA berjalan lambat karena kebutuhan explainability.
- **Self-Driving Car Object Detection (Real-Time ML):** Tesla Autopilot, Waymo, dan Mobileye memakai YOLO/SSD untuk deteksi objek real-time dengan trade-off latency edge vs cloud di bawah 100ms.

Pola yang Sama di Semua Analogi: Tiga elemen kunci yang sama di semua analogi: kualitas data berlabel, feature engineering yang relevan, dan kompleksitas model yang sesuai kebutuhan; kualitas data jauh lebih menentukan daripada pemilihan algoritma semata.

APLIKASI ML PREDICTIVE MAINTENANCE DI INDUSTRI 4.0 & SMART MANUFACTURING

- **Pratt & Whitney Engine Health Management:** 5000+ mesin PW1100G/A320neo dipantau via edge ETL dan ensemble RF/XGBoost/LSTM untuk prediksi Remaining Useful Life; 80% kegagalan terdeteksi 30+ hari lebih awal, menghemat \$5-10 juta per AOG yang dihindari.
- **Siemens Mobility Train Predictive Maintenance:** platform Railigent memakai Random Forest untuk klasifikasi kondisi bogie, motor traksi, gearbox, dan rem; degradasi bearing terdeteksi 2-4 minggu lebih awal, terbukti pada 600+ trainset dengan availability naik 15% dan MTBF naik 30%.
- **GE Predix APM:** 50.000+ aset terhubung dengan digital twin dan deteksi anomali ML untuk komponen hot gas path (umur maksimal 24.000 jam), menurunkan downtime 5-15%.

- **Tesla Battery Management ML:** 7104+ sel baterai per mobil dipantau real-time dengan gradient boosting dan neural network, update model mingguan via OTA, mendukung garansi 8 tahun/240.000 km dengan penurunan kapasitas di bawah 30%.

Peringatan, Jebakan Praktis ML Predictive Maintenance: Delapan jebakan praktis ML predictive maintenance di industri: data berlabel tidak cukup; mengabaikan class imbalance; concept drift tidak dipantau; data leakage; deploy model kotak-hitam tanpa explainability; tidak melakukan A/B test sebelum full rollout; mengabaikan MLOps (versioning, monitoring, retraining); serta ML dianggap menggantikan rule-based padahal seharusnya saling melengkapi.

IMPLEMENTASI PYTHON: MACHINE LEARNING KLASIFIKASI DI JUPYTER NOTEBOOK

Empat cell berikut membangun classifier bearing fault dari nol. Lingkungan: conda env optoauto dengan paket numpy, pandas, matplotlib, scikit-learn; notebook p13_ml_classification.ipynb.

p13_ml_classification.ipynb – Cell 1

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

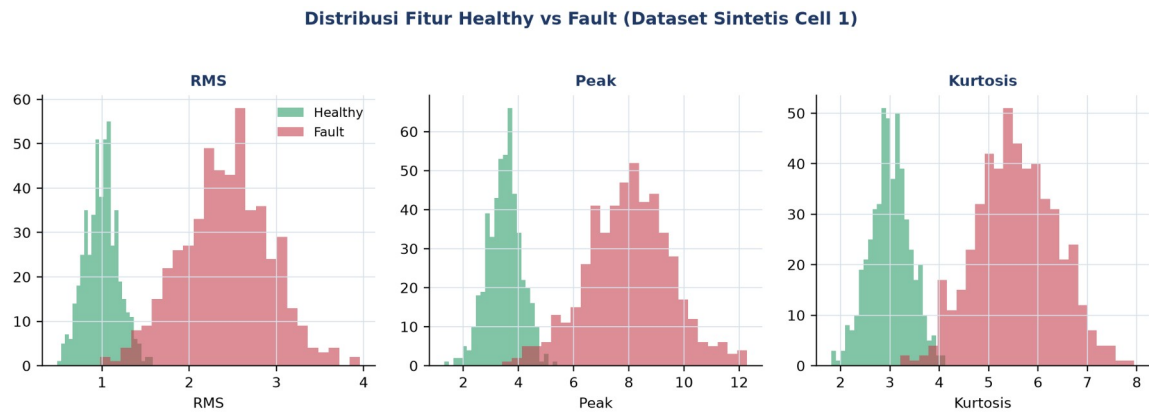
scaler = StandardScaler().fit(X_train)
X_train_s, X_test_s = scaler.transform(X_train), scaler.transform(X_test)

model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train_s, y_train)

print(f'Accuracy: {model.score(X_test_s, y_test):.3f}')
print(f'Koefisien (log-odds): {model.coef_[0]}')
```

Gambar 6. Potongan kode train/test split stratified dan training Logistic Regression baseline.

Gambar 6 menunjukkan potongan kode inti Cell 1.



Gambar 7. Distribusi tiga fitur (RMS, Peak, Kurtosis) untuk kelas healthy vs fault pada dataset sintetis Cell 1; overlap minimal menunjukkan fitur cukup diskriminatif.

Gambar 7 menunjukkan distribusi fitur healthy vs fault pada dataset sintetis 1000 sampel (500/kelas) yang dipakai sepanjang Cell 1-4.

- **Cell 1:** generate dataset sintetis 1000 sampel (RMS, Peak, Kurtosis, seed=42);
``train_test_split(test_size=0.2, random_state=42, stratify=y)``; ``StandardScaler`` fit di train saja; ``LogisticRegression(max_iter=1000)``; cetak accuracy, F1, `classification_report`, dan koefisien.
- **Cell 2:** ``RandomForestClassifier(n_estimators=100, random_state=42)`` dilatih pada data mentah (tanpa scaling); cetak confusion matrix dan ``feature_importances_`` terurut; preview singkat materi Pertemuan 15.
- **Cell 3:** ``Pipeline([StandardScaler, SVC(kernel="rbf", probability=True)])``; perbandingan ROC-AUC tiga model (LogReg, RF, SVM); preview materi Pertemuan 15.
- **Cell 4:** ``cross_val_score(cv=5, scoring="f1_macro")`` dan ``GridSearchCV`` untuk tuning Random Forest; preview materi Pertemuan 15.

TUGAS PERTEMUAN 14

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin.

Distribusi Bobot Tugas Pertemuan 14



Gambar 8. Distribusi 50 poin Tugas Pertemuan 14 berdasarkan tipe soal.

Gambar 8 merangkum distribusi bobotnya.

Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Pendekatan supervised learning berbeda dari unsupervised learning karena...

- A. Supervised butuh GPU, unsupervised tidak
- B. Supervised butuh data labeled (X, y), model belajar mapping input ke output dari contoh. Unsupervised hanya X (cari pattern intrinsik: clustering, dimensionality reduction)
- C. Supervised hanya untuk regresi, unsupervised untuk klasifikasi
- D. Tidak ada perbedaan signifikan

2. Tujuan train_test_split dengan stratify=y di scikit-learn adalah...

- A. Mempercepat training
- B. Menjaga proporsi tiap kelas sama di train dan test set, penting untuk imbalanced data (misal 1% fault). Tanpa stratify, test set bisa kebetulan tidak punya sampel positif sama sekali
- C. Mengubah label jadi numerik
- D. Menghapus duplicate sample

3. Logistic Regression memprediksi probabilitas kelas positif dengan cara...

- A. Menghitung jarak Euclidean ke centroid kelas
- B. Menerapkan fungsi sigmoid pada kombinasi linier fitur: $P(y=1|x)=\sigma(wTx+b)$, menghasilkan nilai 0-1
- C. Membangun banyak decision tree lalu voting
- D. Mengurutkan data dan mengambil median

4. Keunggulan utama Logistic Regression dibanding model black-box (misal deep neural network) adalah...

- **A.** Selalu lebih akurat pada semua jenis data
- **B.** Koefisien w dapat diinterpretasikan langsung sebagai log-odds tiap fitur, memberi interpretability tinggi yang penting untuk domain teregulasi
- **C.** Tidak butuh data untuk training
- **D.** Tidak pernah overfitting

5. Untuk dataset imbalanced 1% fault, 99% healthy, accuracy 99% dari naive predict-all-healthy adalah...

- **A.** Excellent, model sukses
- **B.** Menyesatkan! Recall=0% (semua fault missed). Untuk imbalanced data, accuracy tidak relevan, pakai F_1 -score, precision, recall, ROC-AUC
- **C.** Tidak bisa diinterpretasi tanpa data lebih
- **D.** Bukti overfit

6. Confusion matrix 2x2 (TN, FP, FN, TP): untuk safety-critical predictive maintenance, metric paling kritis adalah...

- **A.** FP (false alarm), yang utama dihindari
- **B.** FN (missed fault), paling berbahaya. Predict "healthy" padahal actual "fault" menyebabkan mesin tidak diperbaiki, catastrophic failure. Solusi: minimize FN via threshold rendah (recall tinggi)
- **C.** TN (correct normal), yang harus dimaksimalkan
- **D.** Semua sama pentingnya

7. F_1 -score = $2 \cdot P \cdot R / (P + R)$ berguna karena...

- **A.** Selalu lebih besar dari accuracy
- **B.** Harmonic mean precision dan recall, single metric yang menyeimbangkan keduanya. F_1 tinggi hanya saat P dan R keduanya tinggi (penalize jika salah satu rendah). Cocok untuk imbalanced data
- **C.** F_1 = accuracy untuk binary classification
- **D.** Hanya bekerja untuk 2 kelas

8. Parameter random_state=42 pada train_test_split dan model scikit-learn berfungsi untuk...

- **A.** Mengatur jumlah fitur yang dipakai
- **B.** Menjamin reproducibility, hasil split/training yang sama setiap kali kode dijalankan ulang
- **C.** Menentukan jumlah kelas target
- **D.** Mempercepat training 42 kali lipat

9. StandardScaler perlu diterapkan sebelum Logistic Regression karena...

- **A.** Logistic Regression tidak bisa menerima angka negatif

- **B.** Gradient descent pada cross-entropy loss konvergen lebih stabil dan cepat saat fitur memiliki skala yang sebanding; scaler harus di-fit hanya di train set untuk mencegah data leakage
- **C.** StandardScaler mengubah masalah klasifikasi jadi regresi
- **D.** Tidak pernah diperlukan untuk model linear

10. Praktik terbaik arsitektur ML predictive maintenance production adalah...

- **A.** ML 100% menggantikan rule-based
- **B.** Hybrid rule-based + ML: rule-based ISO 10816 (Pertemuan 12-13) sebagai safety net dan compliance; ML (Pertemuan 14) sebagai early-warning dan diagnosis multi-fitur. Dashboard operator terpadu, model di-retrain berkala
- **C.** Hanya ML, abaikan rule-based
- **D.** Hanya rule-based, ML tidak diperlukan

Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

Catatan: confusion matrix $[[TN, FP], [FN, TP]]$; $accuracy = (TP + TN) / N$; $precision = TP / (TP + FP)$; $recall = TP / (TP + FN)$; $F_1 = 2PR / (P + R)$.

C1. Hitung accuracy dari confusion matrix: $TP=80$, $FP=10$, $TN=85$, $FN=5$. Formula: $(TP + TN) / (TP + FP + TN + FN)$. Print persentase desimal (misal 91.67).

-  **HINT:** Accuracy = $(TP + TN)$ dibagi $(TP + FP + TN + FN)$, lalu kalikan 100 untuk persen. Substitusikan nilai confusion matrix dari soal.

C2. Hitung precision dari confusion matrix: $TP=80$, $FP=10$. Formula: $TP / (TP + FP)$. Print desimal (3 digit).

-  **HINT:** Precision = TP dibagi $(TP + FP)$, proporsi prediksi positif yang benar. Substitusikan nilai dari soal.

C3. Hitung recall (sensitivity): $TP=80$, $FN=5$. Formula: $TP / (TP + FN)$. Print desimal (3 digit).

-  **HINT:** Recall = TP dibagi $(TP + FN)$, proporsi positif aktual yang terdeteksi. Substitusikan nilai dari soal.

C4. Hitung F_1 -score dari $precision=0,750$ dan $recall=0,600$. Formula: $2.P.R / (P + R)$. Print desimal (3 digit).

-  **HINT:** $F_1 = 2.P.R$ dibagi $(P + R)$, harmonic mean precision dan recall. Substitusikan P dan R dari soal.

C5. Untuk dataset $N=1000$ sampel dengan $test_size=0.2$, berapa sampel di training set? Print integer.

-  **HINT:** Training set = $N \times (1 - test_size)$; sisanya menjadi test set. Substitusikan N dan $test_size$ dari soal.

C6. 5-fold Cross-Validation: untuk dataset 800 sampel, berapa sampel di tiap fold sebagai test? Print integer.

-  **HINT:** Ukuran fold test = total sampel dibagi jumlah fold (k); sisanya jadi train tiap iterasi. Substitusikan nilai dari soal.

C7. Untuk Random Forest dengan `n_estimators=100` dan `max_features='sqrt'` pada dataset dengan 16 fitur, berapa fitur dipertimbangkan per split? Print integer (round).

-  **HINT:** Dengan `max_features='sqrt'`, jumlah fitur per split = $\sqrt{(\text{total fitur})}$, dibulatkan. Substitusikan jumlah fitur dari soal.

C8. StandardScaler: untuk fitur dengan `mean=5,0`, `std=2,0`, hitung scaled value dari `raw=8,0`. Formula: $z=(x-\text{mean})/\text{std}$. Print desimal.

-  **HINT:** Scaled value $z = (x-\text{mean})$ dibagi std. Substitusikan raw, mean, dan std dari soal. Standardisasi wajib untuk SVM/logistic, tidak untuk tree-based.

C9. GridSearchCV: untuk `param_grid` dengan `n_estimators: [50, 100, 200]` dan `max_depth: [None, 5, 10, 20]`, berapa total kombinasi hyperparameter? Print integer.

-  **HINT:** Total kombinasi = perkalian jumlah nilai tiap hyperparameter di grid. Substitusikan jumlah nilai tiap parameter dari soal.

C10. Hitung FNR (False Negative Rate) = missed fault rate, dari `TP=80`, `FN=5`. Formula: $\text{FN}/(\text{TP}+\text{FN})$. Print persentase desimal.

-  **HINT:** $\text{FNR} = \text{FN}$ dibagi $(\text{TP}+\text{FN})$, kalikan 100 untuk persen; ini komplemen recall ($\text{FNR}=1-\text{Recall}$). Substitusikan nilai dari soal.

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan 20-50+ baris kode dengan algoritma lengkap dari awal. Hint minimal. Jawaban benar: +4 poin. Jawaban salah (kode ada tapi hasil tidak sesuai/error): +1 poin partial. Kode kosong = 0 poin (bisa retry).

C11 (Hard). Logistic Regression Baseline, Synthetic Bearing Dataset: generate dataset 2-class healthy vs fault dengan separasi jelas. `np.random.seed(42); n=500; X_h=np.random.randn(n,3); X_f=np.random.randn(n,3)+2.0; X=np.vstack([X_h,X_f]); y=np.concatenate([np.zeros(n),np.ones(n)])`. Split 80/20 stratified, train LogisticRegression, print test accuracy (3 desimal).

-  **HINT:** Split data 80/20 stratified dengan `train_test_split`, latih LogisticRegression, lalu evaluasi dengan `.score()` pada test set.

C12 (Hard). Random Forest + 5-Fold Cross-Validation: generate dataset healthy vs fault. `np.random.seed(42); n=300; X_h=np.random.randn(n,4); X_f=np.random.randn(n,4)+2.0; X=np.vstack([X_h,X_f]); y=np.concatenate([np.zeros(n),np.ones(n)])`. Train

RandomForestClassifier(n_estimators=100, random_state=42) dengan 5-fold CV, print mean accuracy (3 desimal).

-  **HINT:** Gunakan cross_val_score dengan RandomForestClassifier, cv=5, scoring accuracy, lalu ambil rata-ratanya dengan .mean().

C13 (Hard). SVM RBF Pipeline + StandardScaler: dataset multi-skala (butuh scaling sebelum SVM). np.random.seed(42); n=250; X_h=np.random.randn(n,5)*10; X_f=np.random.randn(n,5)*10+5; X=np.vstack([X_h,X_f]); y=np.concatenate([np.zeros(n),np.ones(n)]). Bangun Pipeline([StandardScaler, SVC(kernel="rbf", C=1.0, gamma="scale", random_state=42)]). Split 75/25 stratified, train, print test accuracy (3 desimal).

-  **HINT:** Bangun Pipeline berisi StandardScaler lalu SVC(kernel="rbf") agar scaling otomatis dan bebas data leakage; split stratified, latih, lalu evaluasi .score() di test set.

C14 (Hard). GridSearchCV, Hyperparameter Tuning RandomForest: dataset 4 fitur. np.random.seed(42); n=250; X_h=np.random.randn(n,4); X_f=np.random.randn(n,4)+2.0; X=np.vstack([X_h,X_f]); y=np.concatenate([np.zeros(n),np.ones(n)]). Tune n_estimators=[50,100], max_depth=[3,5,None] dengan GridSearchCV(cv=3, scoring="accuracy"). Print best_score_ (3 desimal).

-  **HINT:** Jalankan GridSearchCV dengan param_grid dan cv dari soal, panggil .fit(), lalu baca skor terbaik dari atribut best_score_.

C15 (Hard). Engineering Pipeline, End-to-End ML Comparison (3 model): dataset bearing fault. np.random.seed(42); n=300; X_h=np.random.randn(n,5); X_f=np.random.randn(n,5)+1.8; X=np.vstack([X_h,X_f]); y=np.concatenate([np.zeros(n),np.ones(n)]). Split 80/20 stratified. Train 3 model: LogReg, RandomForest(n=100), SVM-RBF Pipeline (Scaler+SVC probability=True). Print best ROC-AUC dari 3 model di test set (3 desimal).

-  **HINT:** Untuk tiap model ambil probabilitas kelas positif via predict_proba(X_test[:,1]), hitung roc_auc_score(y_test, proba), lalu pilih AUC terbaik. SVM perlu probability=True.

DAFTAR PUSTAKA

- Zhang, M., Guo, J., Li, X., & Jin, R. (2020). Data-driven anomaly detection approach for time-series streaming data. *Sensors*, 20(19), 5646. <https://doi.org/10.3390/s20195646>
- Altaf, M., Akram, T., Khan, M. A., Iqbal, M., Ch, M. M. I., & Hsu, C.-H. (2022). A new statistical features based approach for bearing fault diagnosis using vibration signals. *Sensors*, 22(5), 2012. <https://doi.org/10.3390/s22052012>

- Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>
- Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972. <https://doi.org/10.3390/app12030972>
- Nguyen, T. T., Φ, B. P., Tran, V. T., Nguyen, V.-T., & Nguyen, V. T. T. (2025). Three-stage optimization of surface finish in WEDM of D2 tool steel via Taguchi design and ANOVA analysis. *Metals*, 15(6), 682. <https://doi.org/10.3390/met15060682>